

Production Hub — סדר פעולות

מסמך עבודה. לסמן ☒ ליד מה שנסגר. לפני הכל: לסיים את מה שהתחלת. לא לפתוח חזית חדשה באמצע תיקון.

שלב 0 — להקים את הפרויקט

עשר דקות. אפשר ורצוי עכשיו, גם עם מידע חלקי. ⌚

- ☐ Projects → New Project → שם (Production Hub)
- ☐ לשדה ההוראות (production-hub-project-instructions.md) להעתיק את
- ☐ להעלות לבסיס הידע:
- ☐ (production-hub-context.md)
- ☐ (CLAUDE.md) (מהריפו)
- ☐ (production-hub-review.md)

זהו. הפערים ב-[TODO] יתמלאו תוך כדי — אין סיבה לחכות להם.

שלב 1 — לאסוף מידע

עשרים דקות. אפס סיכון — הכל קריאה בלבד, שום דבר לא משתנה. אפשר לעשות את זה גם באמצע תיקונים אחרים.

1.1 סכמות (crew.json) ו-(tasks.json)

סשן חדש ב-Claude Code:

```
Read one record from server/data/crew.json and one from
server/data/tasks.json.
Output a markdown section documenting the schema of each: field name, type,
and
a one-line note. Replace every personal value (names, phones, emails, rates)
with
a placeholder like <CREW_NAME> - I'm pasting this into a shared doc.

Then check both files for the same class of problems found in shows.json:
fields that are sometimes one type and sometimes another, fields that
duplicate
information stored elsewhere, and empty-string-vs-missing inconsistency. List
what you find under a "Things that need a decision" heading.

Match the style of section 3 in CLAUDE.md. Output markdown only.
```

☐ הורץ

☐ הפלט הודבק לצ'אט → נכנס לסעיף 3

Trace three workflows through the actual code and write them up as markdown. For each: what triggers it, which files and routes are involved in order, what data gets read and written, and what the user sees.

1. Crew assignment – how does the system determine who is available for a show?

Trace from the UI through to the calendar router. If there's no availability

logic at all, say so plainly rather than describing what it would look like.

2. Producing the coordination sheet / Brief PDF – what fields from the show record

actually appear on it, which route generates it, and how does it reach the crew

(download, email, Drive link)?

3. Task lifecycle – tasks exist both embedded in show records and in tasks.json.

Which is authoritative? Are they synced?

Base this only on what the code does. Where behaviour depends on something not in the code, mark it [needs Zlil's answer] instead of guessing.

☐ הורץ

☐ הפלט הודבק לצ'אט → נכנס לסעיף 4

1.3 מה שרק את יכולה לענות

לא צריך קוד. פשוט לכתוב תשובה ולהדביק לצ'אט:

☐ שם הריפו ב-GitHub (סעיף 1)

☐ הסעות — מה המקור האמיתי, `(transportation)` החופשי או `(transportMode)`

☐ `(transportDriver)` המובנים? (סעיף 3) `(transportTime)`

☐ דף תיאום — מי מקבל אותו, איך הוא מגיע אליהם, ומה חייב להופיע בו? (סעיף 4)

☐ שיבוץ — איך את יודעת מי פנוי היום, גם אם המערכת לא יודעת? מה קורה כשמישהו מסרב? (סעיף 4)

☒ **test runner** — נסגר. חוב שנדחה במודע. להוסיף לסעיף 5

No test runner — deliberately deferred, not a decision against testing. Start with the pure functions in `userData.js` before any structural refactor. Don't chase coverage.

שלב 2 — קריטי

כאן נגמר החלק חסר הסיכון. מכאן והלאה, לכל תיקון:

commit → נקי לפני → שינוי אחד בכל פעם → להריץ את האפליקציה ולבדוק ידנית commit

אין בדיקות אוטומטיות שיתפסו רגרסיה. הידיים שלך הן הבדיקה.

2.1 גיבוי — הכי דחוף ברשימה

השאר ברשימה גורמים לבאגים שאפשר לתקן. זה גורם לאובדן.

Is there any automated backup of DATA_DIR? Trace what creates the backups that server/scripts/restore-data.js restores from. If nothing creates them, tell me what the options are for this setup (Railway volume, no DB) – don't implement anything yet, just lay out 2-3 approaches with tradeoffs.

☐ הורץ

☐ קראתי את הממצאים

☐ בחרתי גישה (אם צריך — לדבר על זה בצ'אט)

☐ מיושם ונבדק בפועל ששחזור עובד

הבדיקה האחרונה חשובה. גיבוי שלא ניסית לשחזר ממנו הוא לא גיבוי.

2.2 הרשאות בין משתמשים ואמנים

Security review of the multi-tenancy layer. For every route that accepts an artistId query param: (1) is there an authorization check that the session user is allowed to access that artistId, or is it trusted blindly? (2) is artistId validated/sanitized before it reaches dataPath()? Check specifically for path traversal (../) and for the literal string __art__ appearing inside a userId or artistId. Report what you find as a list – do not fix anything yet.

☐ הורץ

☐ קראתי

☐ תוקן מה שצריך

Look at every place that writes a JSON data file (writeJsonAndCache and any direct fs writes). Two questions: (1) is the write atomic – temp file + rename – or does it truncate and write the real file in place? (2) is there anything preventing two concurrent writes to the same file, given the three cron jobs (gmail-poll, notifications, automations) run alongside user requests? Show me the actual write implementation.

☐ הורץ

☐ קראתי

☐ תוקן מה שצריך

שלב 3 – חשוב

3.1 חוזה הקאש

Audit the cache contract across all routers. Find (a) any code path that writes a JSON data file without calling writeJsonAndCache or invalidate afterwards, and (b) any place that mutates an object returned by readJsonCached in place (push, splice, direct property assignment) instead of copying first. List them by file and line.

☐ הורץ · [] תוקן

3.2 פולימורפי `schedule`

The schedule field on shows is sometimes a string and sometimes an array of {time, activity}. First: count how many records of each shape exist across all shows.json files under DATA_DIR. Then find every place in the code that reads schedule and tell me which ones handle both shapes and which assume one. Don't migrate anything yet.

☐ הורץ · [] הוחלט על צורה אחת · [] מיגרציה

A show has crewIds, crewEmails, and technicalCrew (free text). Trace where each one is written and where each one is read – especially which one the coordination sheet / brief PDF actually renders from. Can they drift apart? Show me the code paths, then tell me whether technicalCrew could be derived from crewIds at render time instead of being stored.

☐ הורץ [] הוחלט

שלב 4 — כשיהיה זמן

- ☐ 4.1 Puppeteer — singleton-עמידות של ה
- ☐ 4.2 App.jsx-מרוץ בהחלפת אמן — ב
- ☐ 4.3 rate limiting, TOTP, תוקף טוקן — הקשחת התחברות

הפרומפטים המלאים בסעיפים 7-9 של (production-hub-review.md).

שלב 5 — לא דחוף

- ☐ 5.1 בדיקות ל- (userData.js) — ארבע פונקציות טהורות, בערך 40 שורות
- ☐ 5.2 פיצול activity-log — (index.js), ואז invitations, ואז team

הפיצול הוא הזזה מכנית בלבד. לא לשפר תוך כדי.

מתי לגעת בזה: כשאת ממילא צריכה לשנות משהו באחד מהאזורים האלה. אז ההעברה כמעט חינם.

כללים לאורך כל הדרך

סשן נפרד לכל סעיף	לא לגרור נושאים. ההקשר מתלכלך והדיוק יורד
קודם דוח, אחר כך תיקון	כל הפרומפטים מבקשים ממצאים. לקרוא, להחליט, ורק אז לבקש תיקון
commit לפני כל שינוי	זו הדרך היחידה לחזור אחורה
בדיקה ידנית אחרי	להריץ, לפתוח הופעה, לייצר דף תיאום
החלטה שהתקבלה → למסמך	אחרת נדון בה שוב בעוד חודשיים